EXPOSICIÓN Y DEFENSA: METODOLOGÍA XP, PRÁCTICAS XP Y DEFENSA DEL EQUIPO

1. PRESENTACIÓN DEL PROYECTO

1.1. Nombre del proyecto

LABCONTROL — Sistema Web para la Gestión y Control de Laboratorios del Centro de Cómputo.

1.2. Descripción

LABCONTROL es una propuesta de sistema web orientada a centralizar y mejorar la gestión de los laboratorios del Centro de Cómputo de Univalle, sede La Paz.

El sistema busca facilitar la consulta de horarios y disponibilidad, la solicitud y aprobación del uso de laboratorios, el registro de utilización de equipos y el reporte y seguimiento de incidencias.

La plataforma está pensada para estudiantes, docentes, encargados del Centro de Cómputo y administradores.

El proyecto se desarrollará progresivamente, comenzando con las funcionalidades esenciales y agregando capacidades adicionales a medida que exista una necesidad justificada.

1.3. Problema que se busca resolver

La gestión de los laboratorios puede depender de información distribuida, registros físicos o consultas presenciales. Esto puede dificultar la consulta de horarios, la identificación de espacios disponibles, el seguimiento de solicitudes y la comunicación de fallas en los equipos.

LABCONTROL pretende centralizar esa información para facilitar el acceso, organizar las solicitudes y mejorar la trazabilidad de las operaciones.

1.4. Visión del proyecto

La visión de LABCONTROL es convertirse en una plataforma web que permita consultar, administrar y controlar los laboratorios del Centro de Cómputo mediante información centralizada, procesos organizados y funcionalidades que evolucionen de acuerdo con las necesidades de sus usuarios.

1.5. Objetivo general

Desarrollar progresivamente un sistema web para gestionar y controlar los laboratorios del Centro de Cómputo, aplicando la metodología XP para organizar el trabajo, priorizar funcionalidades, obtener entregas incrementales y mejorar continuamente la calidad del producto.

1.6. Objetivos específicos

- Identificar las necesidades de los usuarios.
- Definir historias de usuario y criterios de aceptación.
- Elaborar y priorizar el Product Backlog.
- Estimar el esfuerzo mediante puntos de historia.
- Organizar el desarrollo en iteraciones.
- Aplicar prácticas de XP para mejorar la comunicación y la calidad.
- Realizar pruebas durante el desarrollo.
- Obtener entregas progresivas y verificar los resultados de cada iteración.

---

2. ¿QUÉ ES UNA METODOLOGÍA ÁGIL?

Una metodología ágil es un enfoque de desarrollo de software que organiza el trabajo en ciclos cortos, favorece la colaboración y permite adaptarse a cambios en los requisitos.

En lugar de esperar hasta el final para mostrar el sistema completo, se busca entregar avances funcionales, recibir observaciones y mejorar el producto continuamente.

Sus características principales son:

- Desarrollo incremental e iterativo.
- Comunicación frecuente.
- Entregas de valor en periodos cortos.
- Participación de las personas interesadas.
- Adaptación ante cambios.
- Evaluación continua de la calidad.
- Revisión frecuente del progreso.

En LABCONTROL, este enfoque permite comenzar con las funcionalidades fundamentales, como el inicio de sesión, los horarios, la disponibilidad y las solicitudes, y ampliar después el sistema.

3. ¿QUÉ ES EXTREME PROGRAMMING (XP)?

Extreme Programming, conocida como XP, es una metodología ágil de desarrollo de software que pone especial énfasis en la calidad técnica, la comunicación, la simplicidad, la retroalimentación y la capacidad de responder a cambios.

XP propone trabajar con ciclos cortos, probar el software continuamente, integrar frecuentemente los cambios y mejorar el código de forma constante.

Su objetivo no es solamente programar rápido, sino desarrollar software útil, comprensible, verificable y fácil de mantener.

3.1. Características de XP

1. **Iterativa:** el trabajo se organiza en ciclos de desarrollo.
2. **Incremental:** cada entrega agrega funcionalidades al producto.
3. **Colaborativa:** el equipo mantiene una comunicación frecuente.
4. **Orientada a pruebas:** las funcionalidades deben comprobarse.
5. **Adaptable:** las prioridades pueden ajustarse según la retroalimentación.
6. **Enfocada en la calidad:** el código se revisa, prueba y mejora.
7. **Centrada en entregas pequeñas:** se procura obtener resultados verificables en lugar de esperar a terminar todo el sistema.

3.2. Valores de XP

Comunicación: los integrantes comparten información, aclaran dudas y coordinan el trabajo.

Simplicidad: se implementa lo necesario para resolver el problema actual, evitando complejidad innecesaria.

Retroalimentación: se revisan los resultados y se utilizan las observaciones para mejorar.

Valentía: el equipo debe ser capaz de corregir errores, modificar decisiones inadecuadas y mejorar el código cuando sea necesario.

Respeto: los integrantes reconocen sus responsabilidades, valoran el trabajo de los demás y colaboran para alcanzar un objetivo común.

3.3. ¿Por qué elegimos XP para LABCONTROL?

XP resulta apropiada para LABCONTROL porque el sistema tiene varias funcionalidades relacionadas entre sí y puede evolucionar conforme se conozcan mejor las necesidades de sus usuarios.

Por ejemplo, primero se puede implementar la consulta de disponibilidad y horarios. Después se puede agregar la solicitud de uso y posteriormente la aprobación por parte del encargado.

De esta manera, el equipo puede revisar cada avance, detectar errores y modificar prioridades sin esperar a que todas las funcionalidades estén terminadas.

---

4. ¿CÓMO SE APLICA XP A LABCONTROL?

La aplicación de XP se organiza alrededor de las necesidades de los usuarios, las historias de usuario, el Product Backlog, las iteraciones, el desarrollo, las pruebas y la retroalimentación.

El proceso general es:

Necesidades del usuario → Historias de usuario → Product Backlog → Priorización y estimación → Planificación de la iteración → Desarrollo → Pruebas → Revisión y retroalimentación → Nueva iteración.

4.1. Historias de usuario

Las historias de usuario describen funcionalidades desde el punto de vista de quien utilizará el sistema.

Una estructura habitual es:

“Como [tipo de usuario], quiero [funcionalidad], para [beneficio].”

Ejemplo de LABCONTROL:

“Como estudiante o docente, quiero consultar la disponibilidad de los laboratorios según la fecha y el horario, para encontrar un espacio disponible sin tener que revisar físicamente cada laboratorio.”

Las historias permiten comprender qué necesita el usuario, por qué lo necesita y cómo se comprobará que la funcionalidad está correctamente implementada.

4.2. Product Backlog

El Product Backlog es la lista priorizada de funcionalidades que se pretende desarrollar en el producto.

En LABCONTROL incluye el inicio de sesión, la consulta de laboratorios, los horarios, la disponibilidad, las solicitudes, su aprobación, la gestión de incidencias y otras funcionalidades evolutivas.

El backlog no significa que todo esté terminado. Es una herramienta para ordenar las necesidades y decidir qué se desarrollará primero.

4.3. Puntos de historia

Los puntos de historia, o Story Points, son una estimación relativa del esfuerzo, la complejidad y la incertidumbre de una historia de usuario.

No equivalen automáticamente a horas de trabajo.

En LABCONTROL, el inicio de sesión se estimó en 8 puntos porque implica validar credenciales y considerar los roles. La consulta de disponibilidad se estimó en 3 puntos porque su alcance inicial es más acotado.

Las estimaciones deben revisarse con el equipo conforme se conozca el esfuerzo real.

4.4. Iteraciones

Una iteración es un ciclo de trabajo en el que el equipo selecciona historias, desarrolla funcionalidades, realiza comprobaciones y revisa los resultados.

Cada iteración debe tener un objetivo claro y producir un resultado verificable.

No basta con asignar historias a una iteración: hay que comprobar cuáles fueron implementadas, cuáles superaron las pruebas y cuáles siguen pendientes.

---

5. PRODUCT BACKLOG DE LABCONTROL

El documento maestro organiza el backlog en 20 funcionalidades distribuidas en cuatro iteraciones propuestas.

ID	Historia o funcionalidad	Prioridad	Puntos	Iteración
HU-01	Inicio de sesión y autenticación	Alta	8	1
HU-02	Consultar disponibilidad de laboratorios	Alta	3	1
HU-03	Consultar horarios	Alta	3	1
HU-04	Consultar información del laboratorio	Media	3	1
HU-05	Solicitar uso de laboratorio	Alta	5	1
HU-06	Consultar mis solicitudes	Media	3	1
HU-07	Consultar estado de solicitud	Media	3	1
HU-08	Gestionar solicitudes: aprobar o rechazar	Alta	5	2
HU-09	Registrar utilización de laboratorios o equipos	Media	5	2
HU-10	Administrar laboratorios	Media	5	2
HU-11	Administrar horarios	Media	5	2
HU-12	Gestionar usuarios y roles	Media	5	2
HU-13	Reportar incidencias	Alta	5	3
HU-14	Consultar estado de incidencias	Media	3	3
HU-15	Gestionar incidencias	Alta	5	3
HU-16	Historial de solicitudes	Baja	5	4
HU-17	Historial de incidencias	Baja	5	4
HU-18	Reportes	Baja	5	4
HU-19	Notificaciones	Media	5	4
HU-20	Indicadores y estadísticas	Baja	5	4

Nota: las prioridades, puntos e iteraciones corresponden a la planificación documentada. Deben tratarse como estimaciones de planificación, no como prueba de que las funcionalidades ya estén implementadas.

5.1. ¿Cómo se justifican las prioridades?

Las historias iniciales permiten construir el flujo principal del sistema.

- La autenticación controla el acceso y los roles.
- La disponibilidad y los horarios ayudan a conocer cuándo se puede utilizar un laboratorio.
- La solicitud permite iniciar la gestión del uso.
- La consulta de solicitudes permite dar seguimiento.
- La aprobación permite al encargado resolver las solicitudes.
- Las incidencias permiten registrar y atender fallas.
- Los historiales, reportes, notificaciones e indicadores amplían la gestión y el análisis.

La prioridad se basa en el valor para el usuario, las dependencias entre funcionalidades y la necesidad de obtener primero un flujo básico de funcionamiento.

5.2. Resumen de puntos de historia

Iteración	Funcionalidades	Total estimado
1	HU-01 a HU-07	28 puntos
2	HU-08 a HU-12	25 puntos
3	HU-13 a HU-15	13 puntos
4	HU-16 a HU-20	25 puntos
Total	**20 funcionalidades**	**91 puntos**

Estos totales son la suma de las estimaciones del backlog. No representan automáticamente el porcentaje de avance ni garantizan que el equipo pueda completar una iteración en una duración determinada.

La duración real debe acordarse considerando la disponibilidad del equipo, la complejidad de las tareas y la experiencia obtenida durante las primeras iteraciones.

---

6. ITERACIONES Y ENTREGABLES

6.1. Iteración 1: flujo básico del usuario

Objetivo: permitir que los usuarios accedan al sistema, consulten información de los laboratorios y realicen solicitudes.

Historias incluidas:

- HU-01: Inicio de sesión.
- HU-02: Consultar disponibilidad.
- HU-03: Consultar horarios.
- HU-04: Consultar información del laboratorio.
- HU-05: Solicitar uso.
- HU-06: Consultar mis solicitudes.
- HU-07: Consultar estado de solicitud.

Entregable esperado: primera versión funcional del flujo básico, con las funcionalidades seleccionadas implementadas y verificadas según sus criterios de aceptación.

6.2. Iteración 2: gestión administrativa

Objetivo: permitir que el personal autorizado administre las solicitudes y mantenga actualizada la información operativa.

Historias incluidas:

- HU-08: Gestionar solicitudes.
- HU-09: Registrar utilización.
- HU-10: Administrar laboratorios.
- HU-11: Administrar horarios.
- HU-12: Gestionar usuarios y roles.

Entregable esperado: incremento funcional que incorpore la gestión administrativa, con controles de acceso y pruebas correspondientes.

6.3. Iteración 3: incidencias

Objetivo: facilitar el registro, consulta y seguimiento de las incidencias de los laboratorios y equipos.

Historias incluidas:

- HU-13: Reportar incidencias.
- HU-14: Consultar estado de incidencias.
- HU-15: Gestionar incidencias.

Entregable esperado: módulo de incidencias integrado y verificable.

6.4. Iteración 4: funciones de apoyo y análisis

Objetivo: incorporar funciones que mejoren el seguimiento histórico y la gestión de información.

Historias incluidas:

- HU-16: Historial de solicitudes.
- HU-17: Historial de incidencias.
- HU-18: Reportes.
- HU-19: Notificaciones.
- HU-20: Indicadores y estadísticas.

Entregable esperado: funcionalidades de apoyo para consultar información histórica, comunicar novedades y analizar el uso del sistema.

6.5. ¿Cómo se mide el avance?

El avance debe medirse mediante historias terminadas y verificadas, no solamente por las horas trabajadas o el número de tareas iniciadas.

Para considerar una historia terminada, el equipo debe comprobar que cumple sus criterios de aceptación, que se integra con los componentes correspondientes y que no presenta errores conocidos que impidan su uso previsto.

Una historia que está programada, pero no probada, debe identificarse como pendiente de verificación y no presentarse automáticamente como terminada.

---

7. ESTADO ACTUAL DEL PROYECTO

El documento maestro de LABCONTROL establece una línea base inicial.

Elemento	Estado de la línea base documental
Problema y usuarios	Definidos
Visión y alcance	Definidos
MVP y Product Backlog	Definidos conceptualmente
Historias de usuario	Versión inicial definida
Modelo de datos	Modelo base definido
Arquitectura	Definida conceptualmente
Tecnologías	Seleccionadas
Desarrollo real del código	Pendiente en la línea base
Configuración formal de Git	Pendiente en la línea base
Pruebas	Pendientes de implementación en la línea base
Despliegue	Previsto para una etapa posterior

Este estado corresponde al documento maestro y debe actualizarse con los avances posteriores.

7.1. ¿En qué iteración se encuentra LABCONTROL?

La documentación inicial consolidó la **Fase 0 de planificación y definición**. No demuestra por sí sola que la Iteración 1 esté terminada.

Por tanto, antes de la exposición, el equipo debe revisar el código que tiene disponible y verificar:

1. Qué funcionalidades están realmente implementadas.
2. Cuáles se pueden ejecutar y demostrar.
3. Qué historias de usuario corresponden a esas funcionalidades.
4. Qué pruebas se han realizado y cuáles faltan.
5. Qué tareas permanecen pendientes.

Si el código disponible permite demostrar funcionalidades de la Iteración 1, se debe identificar ese avance con precisión. Si únicamente existe una pantalla de inicio de sesión o un prototipo, se debe presentar como tal.

No se debe afirmar que el proyecto está al 40–50 % sin una medición sustentada en historias implementadas, criterios de aceptación y evidencias.

7.2. ¿Qué se debe mostrar durante la demostración?

La demostración debe relacionar el sistema visible con el backlog y las iteraciones.

Por ejemplo:

- Si se muestra el inicio de sesión, relacionarlo con HU-01.
- Si se consulta la disponibilidad, relacionarlo con HU-02.
- Si se visualizan horarios, relacionarlo con HU-03.
- Si se registra una solicitud, relacionarlo con HU-05.
- Si se consultan solicitudes del usuario, relacionarlo con HU-06.

Solo se deben presentar las funciones que estén disponibles. Las demás pueden mostrarse en el backlog como trabajo pendiente.

---

8. PRÁCTICAS DE XP APLICADAS AL PROYECTO

8.1. Small Releases: entregas pequeñas

Consiste en producir versiones pequeñas y funcionales del sistema, en lugar de esperar a implementar todo el producto.

En LABCONTROL, la primera entrega busca cubrir el acceso, la consulta de información y las solicitudes. Las iteraciones posteriores amplían las capacidades administrativas y de incidencias.

Beneficio: permite revisar el producto temprano y detectar problemas antes de que se acumulen.

8.2. Continuous Integration: integración continua

Consiste en integrar los cambios del equipo frecuentemente y comprobar que el sistema siga funcionando.

En la propuesta de LABCONTROL se contempla utilizar Git y GitHub, con automatizaciones progresivas mediante GitHub Actions.

La integración continua no se demuestra únicamente porque el código esté guardado en GitHub. Debe existir una práctica real de integración frecuente y comprobaciones de los cambios.

Beneficio: reduce conflictos de integración y permite detectar errores antes.

8.3. Pair Programming: programación en parejas

Consiste en que dos integrantes trabajen juntos sobre una tarea de programación, compartiendo la resolución del problema y revisando el código.

En LABCONTROL puede aplicarse especialmente en funcionalidades críticas, como la autenticación, la gestión de permisos o la validación de solicitudes.

Beneficio: favorece el aprendizaje compartido y permite detectar errores durante el desarrollo.

8.4. Collective Ownership: responsabilidad colectiva del código

Significa que la calidad y el mantenimiento del código son responsabilidad del equipo, aunque existan integrantes con funciones principales diferentes.

Por ejemplo, si una persona desarrolla una funcionalidad y otra detecta un error, el equipo debe colaborar para corregirlo.

Beneficio: evita que el conocimiento de una parte esencial del sistema dependa de una sola persona.

8.5. Refactoring: refactorización

Es mejorar la estructura interna del código sin cambiar el comportamiento esperado de la funcionalidad.

En LABCONTROL puede utilizarse para eliminar duplicaciones, mejorar nombres, separar responsabilidades y facilitar el mantenimiento.

Beneficio: mantiene el código comprensible y reduce la dificultad de realizar cambios posteriores.

8.6. Testing: pruebas

Consiste en comprobar que las funcionalidades se comporten como se espera y que los cambios no introduzcan errores.

Ejemplos:

- Verificar que un usuario con credenciales correctas pueda acceder.
- Comprobar que una contraseña incorrecta no permita iniciar sesión.
- Verificar que la consulta muestre la disponibilidad correspondiente.
- Impedir que una solicitud se registre sin los campos obligatorios.
- Comprobar que un usuario solo consulte sus propias solicitudes.

Beneficio: aumenta la confianza en el sistema y ayuda a detectar defectos antes de la entrega.

8.7. Feedback: retroalimentación frecuente

Consiste en revisar los avances con el equipo y, cuando sea posible, con usuarios o responsables del Centro de Cómputo.

La retroalimentación permite confirmar si las funcionalidades responden a las necesidades reales y ajustar las prioridades.

Beneficio: evita desarrollar funcionalidades que no aporten suficiente valor o que no respondan a las necesidades del usuario.

8.8. Documentación viva

La documentación debe actualizarse cuando cambian las historias, las decisiones técnicas, la arquitectura o el comportamiento del sistema.

En LABCONTROL, los cambios relevantes deben reflejarse en las historias de usuario, el backlog, los diagramas y el README.

Beneficio: todos los integrantes trabajan con información coherente y pueden comprender la evolución del producto.

9. HERRAMIENTAS Y TECNOLOGÍAS

9.1. Herramienta de gestión

El proyecto contempla Jira Software para organizar las historias de usuario, las tareas, los responsables y el seguimiento del trabajo.

El tablero debe reflejar el estado real de las actividades. Un ejemplo de columnas para organizar el trabajo es:

- Por hacer.
- En desarrollo.
- En revisión o pruebas.
- Terminado.

Una tarea solo debe pasar a “Terminado” cuando cumpla las condiciones acordadas por el equipo.

Si Jira todavía no está configurado o actualizado, se debe indicar ese estado y no presentar capturas de un tablero que no refleje el trabajo real.

9.2. Tecnologías seleccionadas en el documento maestro

Área	**Tecnología**
Frontend	React + TypeScript
Backend	C# + ASP.NET Core Web API
Acceso a datos	Entity Framework Core
Base de datos	Microsoft SQL Server
Autenticación y autorización	ASP.NET Core Identity y/o JWT, según el diseño definitivo
Control de versiones	Git
Repositorio	GitHub
Integración continua	GitHub Actions
Documentación	Markdown, README, diagramas y documentos
Pruebas	Herramientas de testing de .NET y pruebas de API/interfaz según las necesidades

Estas tecnologías corresponden a la selección documentada. Antes de la demostración, el equipo debe verificar que el código que presentará realmente utiliza esa arquitectura. Si el código recuperado pertenece a otro prototipo o emplea tecnologías distintas, hay que aclarar qué se está mostrando y cuál es la arquitectura objetivo.

10. PLAN DE PRUEBAS

Las pruebas son importantes porque permiten comprobar si el sistema satisface los requisitos y criterios de aceptación.

10.1. Pruebas de autenticación

- Acceder con credenciales válidas.
- Rechazar credenciales incorrectas.
- Validar campos obligatorios.
- Comprobar el acceso según el rol del usuario.

10.2. Pruebas de disponibilidad y horarios

- Consultar laboratorios para una fecha y horario.
- Mostrar laboratorios disponibles y no disponibles.

- Consultar horarios de un laboratorio.
- Comprobar que los horarios correspondan al laboratorio seleccionado.

10.3. Pruebas de solicitudes

- Registrar una solicitud válida.
- Rechazar solicitudes con campos obligatorios vacíos.
- Verificar la disponibilidad antes de registrar la solicitud.
- Comprobar que la solicitud quede almacenada correctamente.
- Consultar las solicitudes del usuario autenticado.
- Impedir que un usuario acceda a solicitudes de otra persona.

10.4. Estado de las pruebas

Los resultados deben clasificarse según las comprobaciones efectivamente realizadas:

- **Aprobada:** el resultado real coincide con el esperado.
- **No aprobada:** el resultado real no coincide con el esperado.
- **Pendiente:** todavía no se ejecutó la prueba.
- **Bloqueada:** no puede ejecutarse por falta de una condición necesaria, como el entorno o una funcionalidad.

Las pruebas no deben marcarse como aprobadas únicamente porque estén planificadas.

---

11. NORMATIVA INTERNA DEL EQUIPO

La normativa interna establece acuerdos para que los integrantes trabajen de forma coordinada.

11.1. Comunicación

Los integrantes deben comunicar sus avances, dificultades y bloqueos oportunamente. Si una tarea no puede completarse, se debe informar antes de la fecha acordada.

11.2. Asignación de tareas

Cada tarea debe tener un responsable principal, un objetivo concreto y un estado actualizado.

La asignación de un responsable no significa que los demás integrantes no puedan ayudar.

11.3. Control de versiones

Los cambios del código deben organizarse mediante Git cuando el repositorio esté configurado. Los mensajes de los commits deben describir los cambios realizados y no se deben sobrescribir los avances de otros integrantes sin coordinación.

11.4. Revisión del código

Antes de considerar terminada una funcionalidad, el equipo debe revisar su comportamiento y comprobar que no afecte negativamente a otras partes del sistema.

11.5. Pruebas obligatorias

Cada historia debe contar con criterios de aceptación claros y comprobaciones acordes con su funcionalidad.

11.6. Documentación

Las modificaciones importantes deben reflejarse en las historias, el backlog y los documentos afectados.

11.7. Cumplimiento de compromisos

Cada integrante debe comprometerse con tareas realistas y comunicar si encuentra dificultades. Los compromisos deben revisarse mediante resultados observables, no únicamente mediante promesas.

11.8. Mejora continua

Al finalizar cada iteración, el equipo debe identificar qué funcionó bien, qué dificultades tuvo y qué cambios realizará en la siguiente iteración.

---

12. ORGANIZACIÓN DEL EQUIPO XP

El documento maestro propone la siguiente distribución de responsabilidades:

Integrante	Rol principal	Responsabilidades
Jared Luis Asturizaga Villarreal	Líder / Coordinación	Coordinación, integración, decisiones y seguimiento.
Jorge Ángel Cores Andrade	Backend / Base de Datos	API, backend, persistencia y modelo de datos.
Dustin Fernando Vaca Aguilar	Frontend / UX-UI	Interfaz, componentes, experiencia de usuario y prototipos.
Jheremy Jhostin Ramirez Arcani	Analista	Requisitos, historias de usuario, casos de uso y análisis funcional.
José Antonio Montes Flores	QA / Tester	Pruebas, validación, evidencias y calidad.

Estos roles son responsabilidades principales, no barreras que impidan colaborar. XP requiere comunicación y responsabilidad compartida.

12.1. ¿Cómo debe organizarse el equipo?

Cada integrante debe conocer:

- Qué funcionalidad se está desarrollando.
- Qué historia de usuario la describe.
- Cuáles son sus criterios de aceptación.
- Quién es el responsable principal.
- Qué depende de otras tareas.
- Qué evidencia permitirá demostrar que se terminó.

El equipo también debe comprobar que la distribución de tareas sea realista y que todos conozcan el avance general.

---

13. DEFENSA DEL EQUIPO XP

Defender el equipo XP significa demostrar que sus integrantes comprenden la metodología, pueden explicar sus decisiones y conocen el estado real del producto.

La defensa no consiste en decir que el equipo utiliza XP porque tiene historias de usuario o un tablero. Debe explicar cómo organiza, desarrolla, prueba, integra y mejora el sistema.

13.1. ¿Por qué utilizamos XP?

Utilizamos XP porque LABCONTROL puede desarrollarse por funcionalidades, permite recibir retroalimentación y requiere comprobar la calidad durante el desarrollo.

13.2. ¿Cómo sabemos qué debemos desarrollar primero?

Priorizamos las funcionalidades según su valor para el usuario, sus dependencias y su contribución al flujo principal del sistema.

Por eso el backlog comienza con el acceso, la disponibilidad, los horarios y las solicitudes, antes de avanzar hacia la gestión de incidencias, los historiales y las estadísticas.

13.3. ¿Cómo sabemos que una iteración terminó?

Una iteración no termina solamente porque transcurrió el tiempo planificado. Se debe revisar qué historias se implementaron, cuáles cumplen sus criterios de aceptación, qué pruebas se realizaron y qué defectos continúan pendientes.

13.4. ¿Qué hacemos si una tarea se retrasa?

Primero comunicamos el bloqueo, revisamos las dependencias y evaluamos el trabajo pendiente. Después acordamos cómo ajustar las tareas o las prioridades, sin ocultar el retraso ni declarar terminada una funcionalidad incompleta.

13.5. ¿Qué hacemos si cambia un requisito?

Analizamos el cambio, su valor y sus consecuencias. Si se acepta, actualizamos la historia de usuario y el backlog, y planificamos su incorporación según las prioridades y la capacidad del equipo.

13.6. ¿Cómo demostramos que aplicamos XP?

Mediante historias de usuario verificables, un backlog actualizado, iteraciones con objetivos claros, entregas demostrables, pruebas, integración frecuente, retroalimentación y acuerdos de trabajo.

La evidencia debe corresponder al trabajo que realmente realizó el equipo.

---

14. GUION PARA LA EXPOSICIÓN ORAL

Introducción

“Buenos días, ingeniera y compañeros. Somos el grupo SQLitos y presentaremos LABCONTROL, un sistema web para la gestión y el control de laboratorios del Centro de Cómputo de Univalle, sede La Paz.

En esta exposición explicaremos el problema que queremos resolver, nuestra visión del producto, la metodología XP, las historias de usuario, las iteraciones, las prácticas de desarrollo y el estado actual del sistema.”

Presentación del problema

“LABCONTROL busca centralizar la información de los laboratorios, facilitando la consulta de horarios y disponibilidad, la gestión de solicitudes y el seguimiento de incidencias.

El propósito es reducir las dificultades que pueden producirse cuando la información está distribuida o depende de procesos manuales y consultas presenciales.”

Explicación de XP

“Elegimos XP como enfoque de desarrollo porque nos permite construir el sistema progresivamente, entregar funcionalidades pequeñas, revisar resultados y corregir errores durante el proceso.

XP enfatiza la comunicación, la simplicidad, la retroalimentación, el respeto, la mejora del código y las pruebas frecuentes.”

Historias de usuario y backlog

“Organizamos las necesidades en historias de usuario. Cada historia describe una funcionalidad desde la perspectiva de quien la utilizará.

Después las priorizamos y estimamos mediante puntos de historia. Esto nos permite ordenar el trabajo y seleccionar las funcionalidades que aportan mayor valor al sistema.”

Iteraciones

“La planificación documentada contempla cuatro iteraciones. La primera se concentra en el flujo básico del usuario; la segunda incorpora la gestión administrativa; la tercera aborda las incidencias y la cuarta incorpora funciones de historial, reportes, notificaciones e indicadores.

Estas iteraciones constituyen nuestra planificación. El estado real de cada una se determina mediante funcionalidades implementadas y verificadas.”

Prácticas XP

“Las prácticas que buscamos aplicar incluyen entregas pequeñas, integración continua, programación en parejas, responsabilidad colectiva del código, refactorización, pruebas y documentación actualizada.

Cada práctica tiene una finalidad: mejorar la coordinación, reducir errores y mantener el sistema comprensible y verificable.”

Estado del sistema

“Presentaremos las funcionalidades que estén disponibles en el código actual y las relacionaremos con sus historias de usuario. También identificaremos los módulos que siguen pendientes, las pruebas realizadas y las tareas que debemos abordar en las siguientes iteraciones.”

Cierre

“En conclusión, XP nos permite organizar el desarrollo de LABCONTROL de manera incremental y orientada a la calidad. Nuestro objetivo es avanzar mediante funcionalidades verificables, mantener la comunicación entre los integrantes y mejorar el producto con base en pruebas y retroalimentación. Muchas gracias.”

---

15. DISTRIBUCIÓN DEL TIEMPO DE EXPOSICIÓN

La exposición debe durar entre 15 y 20 minutos, seguida de 3 a 5 minutos para preguntas.

Parte	Contenido	Tiempo aproximado
1	Presentación del proyecto y problema	2 min
2	Visión y objetivos	1 min
3	Metodologías ágiles y XP	3 min
4	Historias de usuario y Product Backlog	3 min
5	Iteraciones, puntos y entregables	3 min
6	Prácticas XP y normativa interna	2 min
7	Demostración del sistema	3 min

8	Estado actual y conclusiones	1 min
	Total aproximado	**18 min**

La duración de la demostración debe ajustarse a las funcionalidades disponibles.

---

16. PREGUNTAS QUE PODRÍA HACER LA DOCENTE

Pregunta 1. ¿Qué es XP?

XP es una metodología ágil de desarrollo de software que enfatiza la calidad técnica, la comunicación, las pruebas, la retroalimentación y las entregas frecuentes.

Pregunta 2. ¿Qué diferencia hay entre desarrollo iterativo e incremental?

El desarrollo iterativo permite revisar y mejorar el producto en ciclos sucesivos. El desarrollo incremental agrega funcionalidades en cada entrega. XP combina ambos enfoques.

Pregunta 3. ¿Qué es una historia de usuario?

Es una descripción breve de una necesidad desde la perspectiva del usuario. Explica quién necesita una funcionalidad, qué quiere hacer y qué beneficio espera obtener.

Pregunta 4. ¿Qué es el Product Backlog?

Es una lista priorizada de funcionalidades que se consideran para el desarrollo del producto. Puede cambiar conforme evolucionan las necesidades y se obtiene retroalimentación.

Pregunta 5. ¿Qué son los puntos de historia?

Son una estimación relativa del esfuerzo, la complejidad y la incertidumbre de una historia. No representan directamente horas de trabajo.

Pregunta 6. ¿Por qué se prioriza el inicio de sesión?

Porque permite controlar el acceso y aplicar permisos según el rol. Es una funcionalidad habilitadora para las funciones que requieren autenticación.

Pregunta 7. ¿Por qué la disponibilidad y los horarios son importantes?

Porque permiten conocer qué espacios pueden utilizarse y en qué momentos. Son necesarios para orientar las solicitudes de uso.

Pregunta 8. ¿Qué es una iteración?

Es un ciclo de trabajo en el que se seleccionan historias, se desarrollan funcionalidades, se realizan pruebas y se revisan los resultados.

Pregunta 9. ¿Qué es una entrega pequeña?

Es una versión limitada pero verificable del producto que aporta una funcionalidad útil y permite obtener retroalimentación antes de desarrollar el resto.

Pregunta 10. ¿Qué es la integración continua?

Es la práctica de integrar cambios frecuentemente y comprobar que el sistema continúa funcionando. No consiste únicamente en guardar código en un repositorio.

Pregunta 11. ¿Qué es la refactorización?

Es mejorar la estructura interna del código sin cambiar su comportamiento esperado.

Pregunta 12. ¿Por qué son importantes las pruebas?

Porque permiten comparar el comportamiento real con los criterios de aceptación y detectar defectos antes de considerar terminada una funcionalidad.

Pregunta 13. ¿Qué sucede si una historia no se termina?

Debe identificarse como pendiente o parcialmente realizada. El equipo revisa el motivo, actualiza el tablero y decide cómo abordarla en la planificación siguiente.

Pregunta 14. ¿Qué sucede si cambian los requisitos?

Se analiza el cambio, se evalúa su valor y se actualiza el backlog si corresponde. La nueva necesidad se planifica según su prioridad y la capacidad del equipo.

Pregunta 15. ¿Qué significa responsabilidad colectiva del código?

Significa que el equipo comparte la responsabilidad de mantener la calidad y el conocimiento del código, aunque cada integrante tenga funciones principales distintas.

Pregunta 16. ¿Qué es la retroalimentación?

Es la información que se obtiene al revisar el producto y escuchar a los usuarios o al equipo. Sirve para identificar mejoras y corregir desviaciones.

Pregunta 17. ¿Cómo sabemos si el sistema está terminado?

Una funcionalidad se considera terminada cuando cumple sus criterios de aceptación, se integra correctamente y pasa las comprobaciones acordadas. El sistema completo se considera terminado cuando satisface el alcance definido y las condiciones de entrega.

Pregunta 18. ¿Por qué utilizar Jira?

Porque permite organizar las historias, las tareas, los responsables y los estados de avance. Su utilidad depende de que el tablero se mantenga actualizado.

Pregunta 19. ¿Qué dificultades tiene el equipo?

La planificación documentada reconoce que deben fortalecerse el seguimiento y la distribución efectiva de las tareas. También es importante validar los requisitos y mejorar las estimaciones con evidencia real.

Pregunta 20. ¿Cuál es el siguiente paso del proyecto?

El siguiente paso es verificar el estado del código disponible, establecer un entorno de trabajo coherente, actualizar el backlog y comenzar o continuar el desarrollo incremental con pruebas y seguimiento de tareas.

---

17. LISTA DE COTEJO: PREPARACIÓN PARA LA DEFENSA

Antes de entregar y exponer, revisar los siguientes puntos:

- [ ] Identificar claramente LABCONTROL y a sus integrantes.
- [ ] Explicar el problema, la visión y los objetivos.
- [ ] Explicar qué es una metodología ágil.
- [ ] Explicar qué es XP y cuáles son sus valores.
- [ ] Justificar las historias de usuario.
- [ ] Presentar el Product Backlog.
- [ ] Explicar las iteraciones, los puntos de historia y los entregables.
- [ ] Indicar la iteración real del sistema, con evidencia.
- [ ] Mostrar Jira o el tablero que se utiliza realmente.
- [ ] Explicar cómo se obtiene retroalimentación.
- [ ] Explicar cómo se integran los cambios.
- [ ] Mostrar el plan de pruebas y sus resultados reales.

- [ ] Presentar la normativa interna del equipo.
- [ ] Demostrar el sistema actual y relacionarlo con las historias.
- [ ] Preparar respuestas para las preguntas de la docente.
- [ ] Llevar la lista de cotejo impresa por proyecto, como indican las instrucciones.

---

18. CONCLUSIÓN

LABCONTROL es una propuesta de sistema web orientada a mejorar la gestión y el control de los laboratorios del Centro de Cómputo. Su planificación contempla funcionalidades de autenticación, consulta de horarios y disponibilidad, solicitudes, gestión administrativa, incidencias y herramientas de seguimiento.

XP proporciona una forma de organizar el desarrollo mediante ciclos cortos, entregas incrementales, pruebas, integración y retroalimentación.

La aplicación de esta metodología exige más que una planificación escrita. Es necesario mantener el backlog actualizado, distribuir las responsabilidades, cumplir los compromisos acordados y comprobar el funcionamiento de las funcionalidades.

Por ello, el equipo debe presentar el avance real de LABCONTROL, relacionarlo con las historias de usuario y explicar qué se encuentra terminado, qué está pendiente y cómo se continuará el desarrollo.

El propósito final es construir un sistema útil, verificable y mantenible, mejorando continuamente el producto y la forma de trabajo del equipo.

---

19. REFERENCIAS BIBLIOGRÁFICAS

- Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley.
- Documento maestro de consolidación del proyecto LABCONTROL, grupo SQLitos, línea base v1.0.
- Documentación de requisitos e historias de usuario de LABCONTROL, Gestión II-2026.
- Material académico de la asignatura Proyecto de Sistemas I sobre metodologías ágiles, planificación de iteraciones y prácticas XP.